

An Operating System (OS) is the foundational software that manages computer hardware and software resources.

It acts as an intermediary between the user of a computer and the physical silicon hardware.

Without an OS, every application would need custom code to handle low-level operations like reading from a disk or drawing to a screen.

The core of any operating system is the kernel, which remains in main memory continuously.

The kernel operates in a privileged state known as kernel mode, giving it unrestricted access to hardware.

User applications run in a restricted user mode to prevent them from crashing or altering critical system structures.

System calls provide the programmatic interface for user applications to safely request services from the kernel.

Modern OS architectures can be broadly categorized into monolithic, microkernel, hybrid, or exokernel designs.

A monolithic kernel houses all core OS services in a single large block of code and memory space for speed.

A microkernel strips the kernel down to basic inter-process communication (IPC) and scheduling, running other services in user space to improve stability.

Process management is one of the OS's most critical responsibilities for keeping the system responsive.

A process is a program in active execution, complete with its own program counter, registers, and memory space.

Process Control Blocks (PCBs) are complex data structures the OS uses to store all information about specific processes.

CPU scheduling determines which process gets control of the processor when there are multiple runnable processes.

The dispatcher is the module that actually gives control of the CPU to the process selected by the short-term scheduler.

First-Come, First-Served (FCFS) is the simplest scheduling algorithm but suffers from the convoy effect where a slow process holds up the queue.

Shortest Job Next (SJN) minimizes average waiting time but requires complex prediction of the next CPU burst length.

Round Robin (RR) scheduling assigns a fixed time quantum to each process, ensuring fairness and preventing starvation.

Priority scheduling assigns a rank to each process, which can lead to starvation if low-priority background tasks are continuously preempted.

Aging is a technique used to resolve starvation by gradually increasing a waiting process's priority over time.

Context switching is the computationally expensive process of saving the old process state and loading the new process state.

OS designers strive to minimize context switches because they represent pure

overhead where no actual application work gets done.

Threads are the smallest sequence of programmed instructions that can be managed independently by a system scheduler.

A single process can contain multiple threads that share the same data and heap but maintain individual execution stacks.

Multithreading allows a single application to perform multiple operations concurrently, vastly improving performance on multi-core hardware.

Concurrency introduces synchronization challenges like race conditions, where the timing of threads affects the final output.

Mutexes (mutual exclusion locks) are used to prevent multiple threads from accessing a shared resource simultaneously.

Semaphores are signaling mechanisms used to control access to common resources by multiple processes in a concurrent environment.

Deadlocks occur when a set of processes are permanently blocked because each is waiting for a resource held by another.

The OS handles deadlocks through prevention, avoidance strategies like the Banker's Algorithm, detection, or by simply ignoring them (the Ostrich algorithm).

Memory management coordinates all system RAM, tracking each byte and deciding which processes get memory and when.